



MIZAN TEPI UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATICS(CCI)
DEPARTMENT OF INFORMATION TECHNOLOGY
COURSE ADVANCED DATABASE SYSTEMS PROJECT
TITLE:FLIGHT BOOKING SYSTEM
COURSE CODE:ITec2072

PREPARED BY GROUP 3

SECTION:A

NAME

ID

- | | |
|--------------------------|---------------|
| 1.Estifanos Getawey..... | MTUUR/8057/17 |
| 2.Asaye Birhan..... | MTUUR/8279/17 |
| 3.Getachew Birhan..... | MTUUR/8018/17 |
| 4.Abule Dulo..... | MTUUR/8968/17 |
| 5.Habtamu Adisu..... | MTUUR/7837/17 |
| 6.Aberham Tadesse..... | MTUUR/7962/17 |
| 7.Daniel Chamasa..... | MTUUR/8881/17 |
| 8.Efram Dachew..... | MTUUR/8928/17 |
| 9.Fenet Tarekegn..... | MTUUR/8387/17 |
| 10.Amlaku Tadele..... | MTUUR/8649/17 |

Submitted to:Mr Birhanu

Submitted Date: 11/09/2018 EC

❖ Table content

page

Acknowledgement.....	1
1.INTRODACTION.....	2
1.1Problem Statement.....	3
1.2.Objectives.....	3
1.3.System Description.....	4
1.4.Requirements.....	4
1.5. Database Design.....	5
1.6. Conceptual Schema And Logical Schema Design Phase.....	6
1.6.1.Entity	6
1.6.2. Attribute:.....	6
1.6.3. Relationship :.....	6
1.7.ER Diagram & Schema Design.....	14
1.8.Unnormalized Form (UNF).....	16
1.8.1.Description.....	16
1.8.2.Identified Problems.....	16
1.9 1NF Flight Table(4 Records).....	17
2. Second Normal form (2NF).....	17
2.1.Rule.....	17
2.2.Change Applied.....	17
2.3.Flight Table--After 2NF.....	18
2.4.Airline Table (Extracted).....	18
3. Third Normal Form (3NF).....	18
3.1.Rule.....	18
3.2.Change Applloed.....	18

3.3.Airport Table (NEW).....	19
3.4.Airline Table (Unchaged).....	19
3.5.Flight Table--Final 3NF.....	19
4.Query Optimization.....	20
5. Transactions & Concurrency.....	20
6. Security & Access Control.....	20
7. Distributed Database Design.....	21
8.Failure Recovery.....	21
9.Methodology.....	22
Conclusion.....	23
References.....	24

Acknowledgement

We would like to express our sincere gratitude to all those who contributed to the successful completion of this project titled "Flight Booking System Database Design."

First and foremost, we thank our instructor for providing us with the opportunity to work on this project and for the valuable guidance, support, and encouragement throughout the development process. His/Her continuous feedback helped us to improve our understanding of database design and its real-world applications.

We are also grateful to our classmates and friends for their cooperation, ideas, and motivation during the preparation of this project.

Finally, we would like to thank our families for their moral support and encouragement throughout our studies.

This project has helped us gain practical knowledge in database systems, including normalization, relational design, and system analysis, which will be valuable for our future academic and professional career.

Introduction

The global aviation sector is a cornerstone of modern infrastructure, requiring sophisticated digital solutions to manage millions of daily transactions. This proposal outlines the framework for a robust Flight Booking System designed to bridge the gap between complex airline operations and user-friendly consumer interfaces. As air travel becomes more accessible, the volume of data generated—from passenger manifests to fuel logistics—necessitates a database that is not just a storage unit, but a high-performance engine. This project focuses on the intersection of scalability and security, ensuring that as the user base grows, the system's responsiveness remains constant. By integrating real-time processing capabilities, the system aims to provide instantaneous updates across various geographical regions simultaneously. We emphasize a modular architecture that allows for easy updates and integration with third-party services like global distribution systems (GDS). The ultimate goal is to create an ecosystem where airlines can optimize their seat inventory while passengers enjoy a seamless booking journey from search to boarding. This introduction serves as a roadmap for developing a system that prioritizes reliability in an industry where every second of downtime can result in significant financial loss. It acknowledges the shift toward mobile-first and cloud-based solutions in the current technological landscape.

1.1.Problem Statement

Current legacy systems in the airline industry often struggle with the sheer velocity of modern web traffic, leading to critical failures. One of the most prevalent issues is the "race condition" in database management, where two users attempt to book the last available seat at the exact same millisecond. Without rigorous concurrency control, this results in double-booking, leading to customer dissatisfaction and operational headaches for airline staff. Furthermore, many existing platforms suffer from "latency spikes" during holiday seasons or promotional sales, causing the system to hang or crash entirely under pressure. Data fragmentation is another hurdle; when information is stored in silos, inconsistencies arise between the booking agent's view and the actual aircraft manifest. Security is perhaps the most pressing concern, as flight databases are prime targets for cyberattacks aimed at stealing sensitive passenger identities and payment details. Many older systems lack modern encryption standards, making them vulnerable to sophisticated breaches. There is also the issue of "orphan records," where a payment is processed but the seat reservation fails to link, creating a vacuum in data integrity. This proposal addresses these gaps by identifying the need for a synchronized, high-speed, and fortified database environment.

1.2.Objectives

The primary ambition of this project is to architect a database that serves as a "single source of truth" for all flight-related operations. A major specific objective is the implementation of strict data constraints at the schema level to make seat duplication mathematically impossible within the system logic. We also aim to achieve "zero-lag" status for availability updates, ensuring that the moment a seat is selected, it is visible as "unavailable" across the entire global network. Security is a

non-negotiable objective, focusing on the integration of end-to-end encryption for all financial transactions and personal passenger data. From a performance standpoint, the objective is to maintain a sub-second response time for 95% of all queries, even during peak load periods. We are also committed to ensuring full compliance with ACID (Atomicity, Consistency, Isolation, Durability) properties, which are the gold standard for reliable transaction processing.

Supporting high concurrency is vital, meaning the system must handle thousands of simultaneous read/write operations without compromising data quality. Additionally, the project seeks to provide a scalable framework that can easily expand from a single regional airline to a multi-national carrier network. Ultimately, these objectives converge to create a system that is resilient, transparent, and highly efficient for all stakeholders.

1.3. System Description

The Flight Booking System operates as a multi-tier application where the database acts as the core intelligence layer. Users interact with the system by inputting search parameters such as city pairs, travel dates, and preferred cabin classes. The system's logic engine then queries the database to return a filtered list of available flights, sorted by price or duration. A key feature of this system is the interactive seat map, which pulls real-time data to show which spots are occupied, restricted, or available for selection. Once a user picks a seat, the system initiates a "soft lock" or temporary reservation, preventing others from choosing it while the payment is being processed. This temporary state is governed by a timer; if the payment isn't confirmed within a set window, the seat is released back into the pool. Upon successful transaction through the integrated gateway, the status shifts to "confirmed," and an e-ticket is generated. The system also includes a comprehensive administrative backend where flight paths can be adjusted and pricing models updated dynamically. For passengers, a "Manage My Booking" portal allows for modifications, cancellations, and meal selections. This holistic approach ensures that every phase of the traveler's lifecycle is captured and managed within a single, unified digital environment.

1.4. Requirements

The functional requirements represent the core actions the system must perform to be useful to its end users. First, it must support a secure registration and profile management system where users can

store their travel preferences and loyalty points. Second, the search functionality must be highly granular, allowing for multi-city itineraries and flexible date searches. Third, the reservation engine must handle the logic of connecting flights, ensuring that layover times are sufficient and baggage is tracked. Fourth, the payment module must interface with various providers, including credit cards, digital wallets, and bank transfers. Fifth, the booking management system must automate the generation of receipts and boarding passes in various formats. On the other hand, non-functional requirements define the quality of the experience. High performance is required to prevent user abandonment during the checkout process. Scalability is essential so the infrastructure can grow alongside the airline's fleet. Security requirements mandate that the system be hardened against SQL injection and cross-site scripting attacks. Reliability ensures the system remains operational of the time, even during maintenance. Finally, the response time for the database must be optimized to ensure that the user interface feels snappy and intuitive, regardless of the complexity of the underlying search query.

1.5. Database Design

The architecture of this database is built upon a foundation of clearly defined entities that mirror real-world aviation components. At the heart of the design are the Passengers and Flights tables, which hold the fundamental data for every transaction. The Airlines table stores carrier-specific information, while the Seats table provides a granular map of every individual spot on every aircraft in the fleet. To link these, a Bookings table acts as a relational bridge, connecting a specific passenger to a specific seat on a specific flight. The Payments table is isolated to ensure that financial records can be audited independently of travel records. Each table is assigned a primary key, typically a unique UUID, to ensure that no two records can ever be confused.

Foreign keys are meticulously mapped to enforce "referential integrity," meaning you cannot have a booking for a flight that each piece of information, such as a passenger's email, is stored in exactly one place that does not exist. We also implement "check constraints" to ensure that values like "Price" or "Seat Number" follow logical formats. To speed up the retrieval of data, we use indexes on frequently searched columns like Origin_City and Departure_Date. This design avoids data redundancy by ensuring t

1.6. Conceptual Schema And Logical Schema Design Phase

Under conceptual phase there is some terminology

1.6.1.Entity :An entity in a database is a thing, place, person or object that is independent of another. In databases, you store information about things in the real

world, and these things are database entities. Types:

A. Weak Entity: represented by double rectangle.

B. Strong Entity: represented by single rectangle.

1.6.2. Attribute: Attribute is a piece of data that describes an entity. Types:

A. simple (atomic) vs. composite entity: Simple contains single value while Composite divide into sub parts. B. Single valued vs. multi valued attribute Single value: have single value and represented by oval shape. Multi value: have more than one shape and represented by double oval.

C. Stored vs. Derived attribute: Stored: not possible to divide. Derived: The value may be derived, represented by dot line oval. D. Null values: The attribute which are not applicable but the value and key attribute cannot be null. E. Default value: Assumed value if not exist value or unknown.

1.6.3. Relationship : a situation that exist between two relational database tables when one table has a foreign key that reference the primary key of the other table.

1.6.4. Identify Entity

- | | |
|--------------|------------|
| 1. Passenger | 5.seat |
| 2. Airline | 6.flight |
| 3. Booking | 7. user |
| 4. Payment | 8.auditlog |

Identify attribute

➤ Passenger

-passid
-Country
-Passport_number
-Name
-Phone_number
-Created_at

➤ Airline

-airid
-Country

flight

-flightid-
-arrival_time
-departure_time
-flightcode
- origin
- destination

➤ **Booking**

-bookid
-Status
-Booking_date

➤ **Payment**

-Paymentid
-Status
-Payment_date
-Method

➤ **Seat**

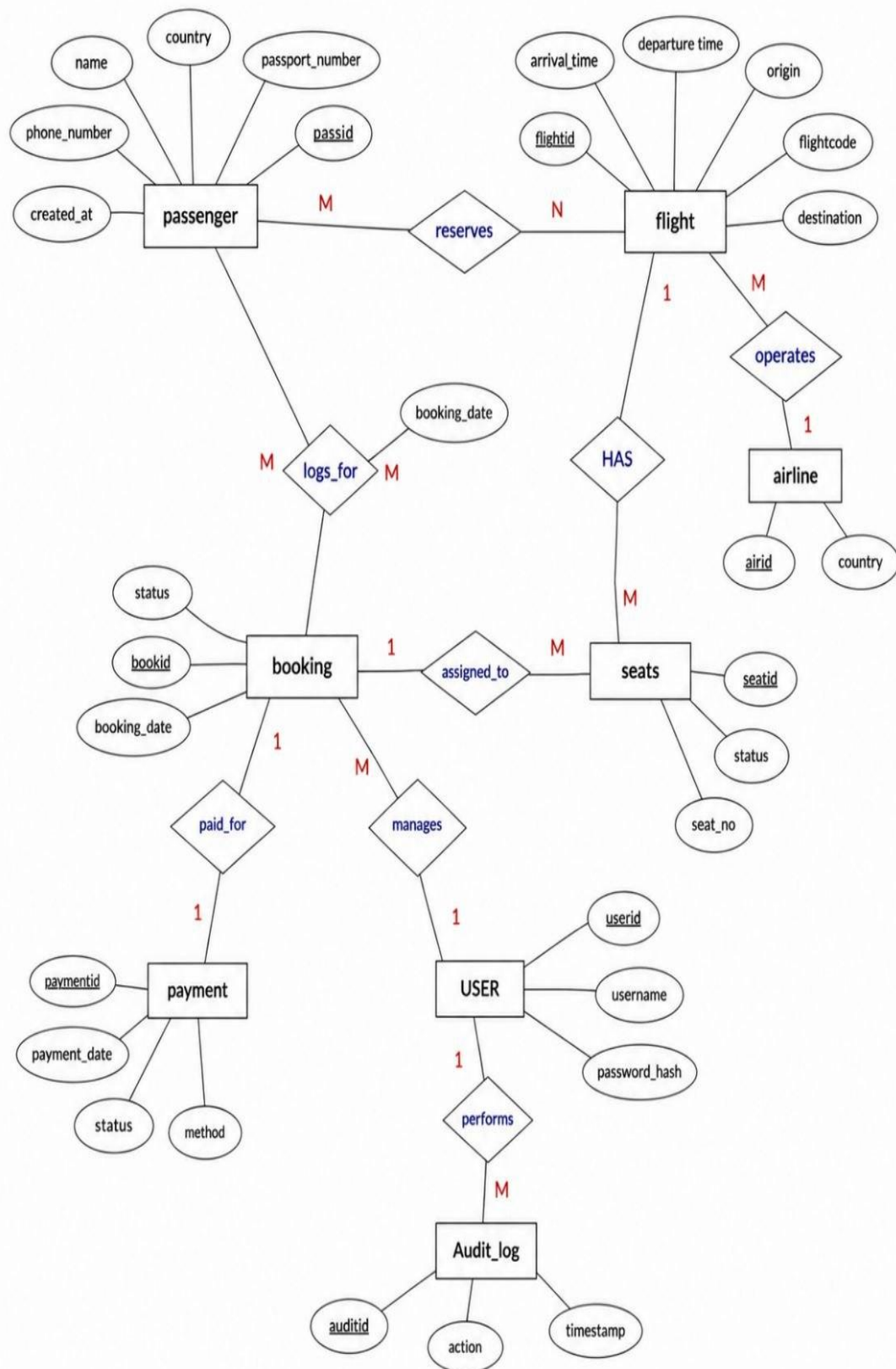
-Seatid
-Status
-Seat_number

➤ **User**

-Userid
-Username
-Password_hash

audit_log

-auditid
-action
-timestamp



```
create database flightbookssystem;
use flightbookssystem;
create table passengers(
passID int primary key auto_increment,
name varchar(20) not null,
country varchar(20) not null,
passport_Number varchar(20) unique not null,
phone_Number int,
createdAt timestamp default current_timestamp
);
```

```
insert into passengers(name,country,passport_Number,phone_Number)
values("asaye", "baharder", "ET987654",0934526715),
("getachew","makele","ET123456",0934590734),("estifanos","dese","ET678654",093
4594575);
select *from passengers;
```

passID	name	country	passport_Numb er	phone_Numb er	createdAt
1	asaye	baharde r	ET987654	934526715	4/13/2026 12:11
2	getache w	makele	ET123456	934590734	4/13/2026 12:11
3	estifanos	dese	ET678654	934594575	4/13/2026 12:11

```
create table Airline(
airID int primary key auto_increment,
country varchar(20) not null
);
insert into Airline(country) values( "ethiopian airlines"),
("emirates airlines"),("kenya airwayes");
select *from Airline;
```

airID	country
1	ethiopian airlines
2	emirates airlines
3	kenya airwayes

```
create table flight(
flightID int primary key auto_increment,
flihtcode varchar(40) not null,
origin varchar(70),
destination varchar(70),
departure_time datetime,
airID int,
arrival_time datetime ,
```

```
foreign key (airID) references airline(airID)
);
```

```
insert into flight(flihtcode,origin,destination,departure_time,arrival_time,airID)
values("ET057", "ethiopia", "keneya","2026-07-09 03:07:55","2026-07-09
08:07:55",1),
("KQ456", "keneya", "nirobi","2026-04-24 03:07:55","2026-04-24 01:07:55",2),
("KQ456", "keneya", "ethiopia","2026-09-24 04:07:20","2026-09-24 01:07:55",3);
select *from flight;
```

flightID	flihtcode	origin	destination	departure_time	airID	arrival_time
1	ET057	ethiopia	keneya	7/9/2026 3:07	1	7/9/2026 8:07
2	KQ456	keneya	nirobi	4/24/2026 3:07	2	4/24/2026 1:07
3	KQ456	keneya	ethiopia	9/24/2026 4:07	3	9/24/2026 1:07

```
create table seat(
seatID int primary key auto_increment,
flightID int,
seat_number varchar(15),
status varchar(30),
unique(flightID,seat_number),
foreign key (flightID) references flight(flightID)
);
```

```
insert into seat (seatID,flightID,seat_number,status) values( 4,1,"seat
1C","Available"),
( 5,2,"seat 2C","Available"),( 6,3,"seat 3C","Booked");
select *from seat;
```

seatID	flightID	seat_number	status
4	1	seat 1C	Available
5	2	seat 2C	Available
6	3	seat 3C	Booked

```
create table booking(
bookID int primary key auto_increment,
flightID int,
passID int,
seatID int,
status varchar(30),
```

```

booking_date timestamp default current_timestamp,
foreign key (flightID) references flight(flightID),
foreign key (passID) references passengers (passID),
foreign key (seatID) references seat(seatID),
unique(flightID,seatID)
);

```

```

insert into booking (paymentID,bookID,flightID,passID,seatID,status) values(7,1,
1,4,"confirmed"),
(8,2, 2,5,"confirmed"),
(9,3, 3,6,"pending");

```

```
select *from booking;
```

bookID	passID	seatID	status	booking_date
7	1	4	confirmed	4/13/2026 12:42
8	2	5	confirmed	4/13/2026 12:42
9	3	6	pending	4/13/2026 12:42

```

create table payment(
paymentID int primary key auto_increment,
bookID int,
method varchar(20),
status varchar(15),
payment_date timestamp default current_timestamp,
foreign key (bookID) references booking(bookID)
);

```

```

insert into payment (paymentID,bookID,method,status)
values(10,7,"telebirr","completed"),
(11,8, "card","pending"),
(12,9, "CBE","pending");

```

```
select *from payment ;
```

paymentID	bookID	method	status	payment_date
10	7	telebirr	completed	4/13/2026 12:51
11	8	card	pending	4/13/2026 12:51
12	9	CBE	pending	4/13/2026 12:51

```

start transaction;
select status from seat where seatID=6 for update;
update seat set status="booked" where seatID=6;
commit;
/*lost update user A*/

```

```

start transaction;
update seat set status="booked" where seatID=6;
/*user B*/
start transaction;
update seat set status="reserved" where seatID=6;
commit;
/*user A */
commit;
select *from seat where seatID=6 for update;
/*lost dirtydata user A*/
start transaction;
update seat set status="booked" where seatID=6;
select *from seat;
/*lost dirtydata user B*/
select *from seat where seatID=6;
/*user A*/
rollback;
select *from seat;
set transaction Isolation level read committed;
/*lost deadlock user A*/
start transaction;
update seat set status="booked" where seatID=4;
/*user B*/
start transaction;
update seat set status="booked" where seatID=6;
/*user A*/
update seat set status="booked" where seatID=6;
/*user B*/
update seat set status="booked" where seatID=4;
show engine InnoDB status;
/*security and access control*/

```

```

/*Admin*/

```

```

create role Admin;
grant all privileges ON passengers to Admin;
grant all privileges ON flight to Admin;
grant all privileges ON seat to Admin;
grant all privileges ON booking to Admin;
grant all privileges ON payment to Admin;
/*Airline staff*/
create role Airline_staff;
grant select,insert,update on flight to Airline_staff;
grant select,update on seat to Airline_staff;
/*customer*/

```



```

create role customer;
grant select ON flight to customer;
grant select ON seat to customer;
grant select ON booking to customer;
grant select ON payment to customer;
/*assign roles to users*/
grant Admin to user1;
grant Airline_staff to user2;
grant customer to user3;
/*password security(hashing)*/

create table user(
userid int primary key,
username varchar(20),
password_hash varchar(200)
);
insert          into          user(userid,username,password_hash)
values(2,"dani",SHA2("mypassword123",256)),
      (3,"abula",SHA2("mypassword456",256)),
      (4,"abrish",SHA2("mypassword701",256));

Select *from user;

userid  username  password_hash
2       dani     6e659deaa85842cdabb5c6305fcc40033ba43772ec00d45c2a3c921741a5e377
3       abula    415858d6b1a389558e5fd3b0b661c223bcee5825b6e75e13e2e9de4e5611d0c2
4       abrish   108796d13303971648ac40ebacdfa9658ba5423e70cce000aa04b64c9902e165

/*encrypt sensitive data (payment)*/
alter table payment modify encrypted_details varbinary(200);
update payment set encrypted_details=aes_encrypt("1234-8765-8907","key431")where
paymentid=1;
/*audit logging*/

create table audit_log(
auditid int primary key auto_increment,
action varchar(30),
userid int,
timestamp timestamp default current_timestamp

);

select *from audit_log;

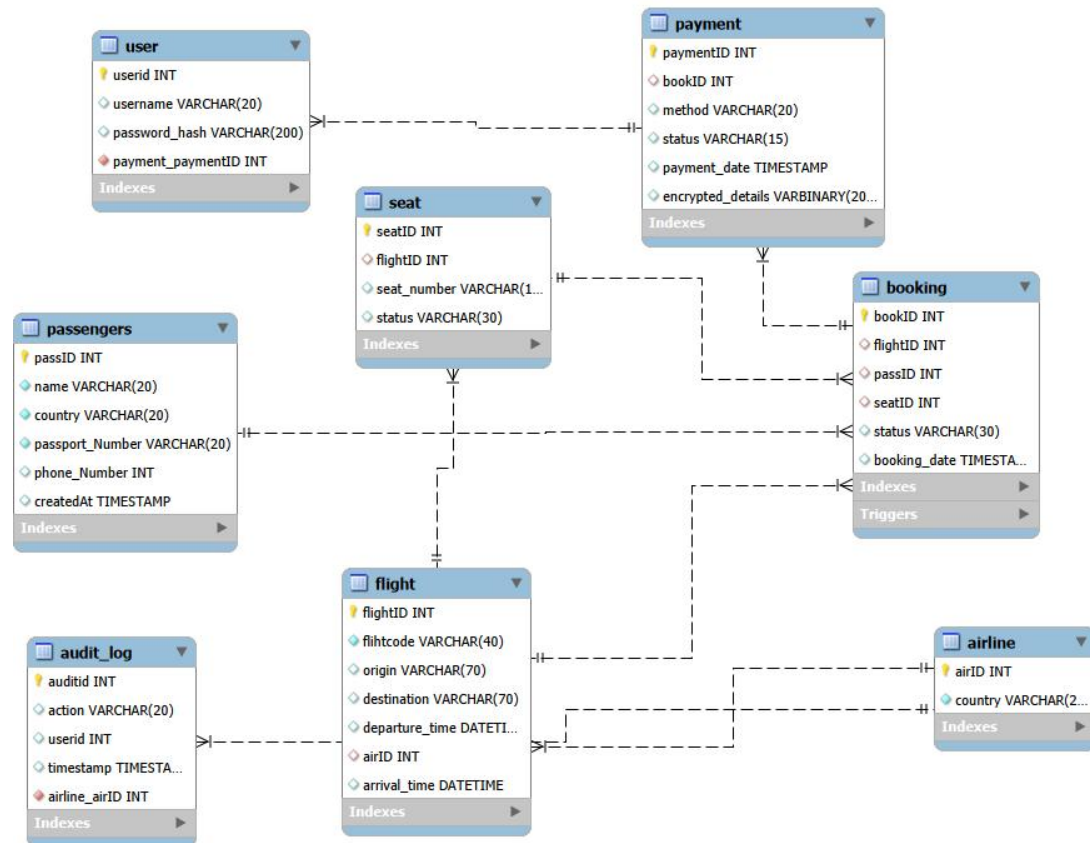
auditid  action                userid  timestamp
1        new booking created    2       4/15/2026 13:10
2        booking cancelled     3       4/15/2026 13:10
3        payment completed   4       4/15/2026 13:10

```

```
/*trigger booking tracking*/  
create trigger log_booking after insert on booking for each row insert into  
audit_log (action,userid)  
values('new booking created', 2),('booking cancelled', 3),('payment  
completed', 4);
```

1.7.ER Diagram & Schema Design

The Entity-Relationship (ER) Diagram serves as the visual blueprint for the entire system, illustrating how different data points interact. In our design, the Flight entity has a "one-to-many" relationship with the Seat entity, as one flight contains many individual seats. Similarly, the Passenger entity has a "one-to-many" relationship with Bookings, allowing a single traveler to manage multiple upcoming trips. The schema design follows the principles of Third Normal Form (3NF) to eliminate "update anomalies" and "redundant data." For instance, instead of storing the airline's phone number in every flight record, we store it once in the Airlines table and reference it via an Airline_ID. This normalization process ensures that if an airline changes its contact info, we only need to update it in one location. We also use a "junction table" for scenarios where many-to-many relationships exist, such as passengers and their various travel documents. The schema includes specific data types for every attribute, such as DATETIME for departures and DECIMAL for currency, to maintain precision. By strictly adhering to these design patterns, the database remains lean and efficient. This structured approach prevents the "data bloat" that often slows down older, poorly designed airline systems. The resulting schema is both easy for developers to understand and optimized for the database engine to execute



1.8. Unnormalized Form (UNF)

1.8.1. Description

The raw Flight data collected before any normalization is applied. The Route column stores two pieces of information (origin and destination) in a single composite field. The Airline column stores a name directly instead of a reference, creating redundancy.

1.8.2. Identified Problems

Violation: Route column stores two values in one field: 'Addis Ababa → Nairobi' — not atomic.

Violation: AirlineName stored directly — same airline name repeated across multiple flight rows. Violation: No separation of concerns — fl

1.8.3. UNF Flight Table (4 Records)

FlightID	FlightNo	origin -> destniction	DateTime	Airline
1	ET101	Addis Ababa → Nairobi	2026-05-01 08:00	Ethiopian Airlines
2	KQ202	Nairobi → Addis Ababa	2026-05-02 14:00	Kenya Airways
3	EK711	Addis Ababa → Dubai	2026-05-03 22:00	Emirates
4	MS961	Nairobi → Cairo	2026-05-04 10:30	EgyptAir

1.9.1NF Flight Table (4 Records)

All values are now atomic. Primary key (FlightID) uniquely identifies each row.

FlightID (PK)	FlightNo	Origin	Destination	DateTime	AirlineID (FK)	AirlineName
1	ET101	Addis Ababa	Nairobi	2026-05-01 08:00	1	Ethiopian Airlines
2	KQ202	Nairobi	Addis Ababa	2026-05-02 14:00	2	Kenya Airways
3	EK711	Addis Ababa	Dubai	2026-05-03 22:00	3	Emirates
4	MS961	Nairobi	Cairo	2026-05-04 10:30	4	EgyptAir

2.Second Normal Form (2NF)

2.1. Rule

Must be in 1NF, and every non-key attribute must be fully functionally dependent on the entire primary key. No partial dependencies are allowed.

2.2.Changes Applied

Violation: AirlineName depends only on AirlineID, not on FlightID. This is a partial dependency causing update anomalies — if an airline renames itself, every flight row must be updated.

Fix: Extracted Airline into its own table with AirlineID (PK) and AirlineName. Flight table retains AirlineID as a Foreign Key. Now all non-key attributes in Flight depend fully on FlightID.

2.3. Flight Table — After 2NF

FlightID (PK)	FlightNo	Origin	Destination	DateTime	AirlineID (FK)
1	ET101	Addis Ababa	Nairobi	2026-05-01 08:00	1
2	KQ202	Nairobi	Addis Ababa	2026-05-02 14:00	2
3	EK711	Addis Ababa	Dubai	2026-05-03 22:00	3
4	MS961	Nairobi	Cairo	2026-05-04 10:30	4

2.4. Airline Table (Extracted)

AirlineID (PK)	AirlineName
1	Ethiopian Airlines
2	Kenya Airways
3	Emirates
4	EgyptAir

Foreign Key Relationship

Flight.AirlineID: → references Airline(AirlineID)

3. Third Normal Form (3NF)

3.1. Rule

Must be in 2NF, and no non-key attribute may depend on another non-key attribute (no transitive dependencies). Every non-key attribute must depend directly on the primary key only.

3.2. Changes Applied

Violation: Origin and Destination store city names directly. If airport details (country, code, timezone) were needed, they would depend on the city name — not on FlightID — creating a transitive dependency.

Fix: Introduced an Airport table. Origin and Destination are replaced by OriginID (FK) and DestinationID (FK) referencing Airport(AirportID). Now all non-key attributes in Flight depend solely on FlightID.

3.3. Airport Table (New)

AirportID (PK)	CityName	Country
1	Addis Ababa	Ethiopia
2	Nairobi	Kenya
3	Dubai	UAE
4	Cairo	Egypt

3.4. Airline Table (Unchanged)

AirlineID (PK)	AirlineName
1	Ethiopian Airlines
2	Kenya Airways
3	Emirates
4	EgyptAir

3.5. Flight Table — Final 3NF

All city name strings replaced with foreign key references to the Airport table.

FlightID (PK)	FlightNo	OriginID (FK)	DestID (FK)	DateTime	AirlineID (FK)
1	ET101	1	2	2026-05-01 08:00	1
2	KQ202	2	1	2026-05-02 14:00	2
3	EK711	1	3	2026-05-03 22:00	3
4	MS961	2	4	2026-05-04 10:30	4

4. Query Optimization

In a high-traffic flight system, the efficiency of SQL queries can be the difference between a smooth booking and a timed-out session. Query optimization begins with the strategic use of "indexes," which act like an index in a book, allowing the database to find data without scanning every single row. We specifically index "Search" fields such as departure airports and dates, as these are the most common entry points for users. Beyond indexing, we use "Query Execution Plans" to identify bottlenecks where the database might be performing "full table scans" instead of targeted lookups. We also practice "query refactoring," which involves rewriting complex joins into more efficient formats to reduce CPU and memory usage. For seat availability, we use "Covering Indexes" so the database can answer the query directly from the index without even touching the main table. Another technique is "materialized views" for historical data, which pre-calculates expensive totals like monthly revenue for administrative reports. We also limit the use of `SELECT *` to ensure we are only pulling the exact columns needed, reducing network overhead. By caching the results of frequent, non-changing queries—like the list of all active airports—we further reduce the load on the database. These optimizations collectively ensure that the system remains lightning-fast even as the number of flights in the database grows into the millions.

5. Transactions & Concurrency

Transactions are the mechanism that ensures a flight booking is an "all or nothing" operation. If a user pays for a ticket but the system fails to update the seat status, the transaction is "rolled back," and the money is refunded to maintain consistency. We use the **BEGIN TRANSACTION**, **COMMIT**, and **ROLLBACK** commands to wrap these critical operations together. Concurrency control is managed through "isolation levels," which determine how visible one user's changes are to another user at the same time. To prevent double-booking, we employ "pessimistic locking" on specific seat rows during the final checkout phase.

This ensures that while one user is typing their credit card details, that specific seat is invisible to all other searchers. We also monitor for "deadlocks," a rare situation where two transactions are waiting for each other to release a lock, by implementing a timeout and retry logic. "Optimistic concurrency control" is used for less critical updates, like changing a passenger's frequent flyer number, where conflicts are unlikely. By managing these simultaneous interactions carefully, we ensure that the database remains stable under the pressure of thousands of concurrent users. This part of the system is what guarantees that the airline never sells the same seat twice. It turns a chaotic environment of global clicks into an orderly, sequential series of confirmed actions.

6. Security & Access Control

Security is implemented through a "defense-in-depth" strategy, starting with Role-Based Access Control (RBAC). In this model, a 'Customer' can only see their own bookings, while 'Staff' can modify flight times, and 'Admins' have full system control. This prevents unauthorized users from accessing sensitive backend data or altering pricing structures. We use "Parameterized Queries" to protect against SQL Injection, ensuring that user input is never executed as code. All passwords are "hashed" using strong algorithms like BCrypt, meaning even if the database is stolen, the actual

passwords remain unreadable. For financial data, we use "Tokenization," where the database stores a reference token instead of the actual credit card number. Data "at rest" is encrypted on the server disks, and data "in transit" is protected by industry-standard TLS encryption. We also maintain "Audit Logs," which record every change made to the database, including who made the change and when. This allows us to track down any suspicious activity or accidental errors made by staff members. Frequent security patches and vulnerability scans are part of the maintenance plan to stay ahead of emerging threats. By treating security as a foundational element rather than an afterthought, we build a system that passengers can trust with their private information.

7.Distributed Database Design

To serve a global audience, the system utilizes a distributed database architecture that spreads data across multiple geographic locations. This "partitioning" or "sharding" of data ensures that a user in London experiences the same speed as a user in New York by accessing a local data node. We use "geographic replication," where the database is cloned across different continents to provide high availability; if one data center goes offline, another takes over instantly. Managing the "CAP theorem" trade-off is central to this design—balancing Consistency, Availability, and Partition Tolerance. For flight availability, we prioritize "Strong Consistency" to avoid overbooking, while for user reviews or flight history, we might allow "Eventual Consistency" to boost performance. We also implement "Global Load Balancing" to direct traffic to the healthiest and closest database server available. This distributed approach also makes the system more "elastic," meaning we can add more servers during high-demand periods like the summer holidays. It reduces the "single point of failure" risk, ensuring that a localized power outage doesn't bring down the global booking network. This sophisticated setup is essential for modern airlines that operate across different time zones and require 24/7 operational capability.

8.Failure Recovery

The failure recovery module is the system's "insurance policy" against hardware crashes, power failures, or human errors. We implement "Write-Ahead Logging" (WAL), which records every change to a log file before it is permanently written to the main database. If the system crashes, it can replay these logs to restore the database to its exact state just before the failure. We also use a "Point-in-Time Recovery" (PITR) strategy, allowing us to "roll back the clock" to a specific minute if a major data corruption occurs.

: Regular "Full Backups" are taken daily, while "Incremental Backups" capture changes every hour to minimize data loss. These backups are stored in multiple, physically separate locations to protect against natural disasters at the primary data center. "Health Checks" and "Auto-Failover" mechanisms are in place to detect a server failure and switch to a "Standby" database in seconds. We also practice "Recovery Drills" to ensure that our team can restore the system within the promised "Recovery Time Objective" (RTO). "Check-pointing" is used to clear out old logs and keep the recovery process fast and manageable. By preparing for the worst-case scenario, we ensure that the airline's data—and the passengers' travel plans—are always protected.

9.Methodology

The development process follows a rigorous "Software Development Life Cycle" (SDLC), starting with an intensive requirement gathering phase. We use an "Agile" approach, breaking the project into two-week "sprints" where specific features like the search engine or payment portal are built and tested. For the database layer, we use MySQL due to its proven track record in handling high-concurrency web applications. "ER Workbench" is our primary tool for visual modeling, allowing us to spot design flaws before a single line of code is written. We implement "Unit Testing" for every database function and "Integration Testing" to ensure the frontend and backend communicate perfectly. "Stress Testing" is conducted by simulating 10,000 simultaneous users to see where the system might break under extreme load. We also use "Version Control" (like Git) for our database scripts, ensuring that every change is tracked and can be reversed if necessary. The methodology includes a "User Acceptance Testing" (UAT) phase where actual airline staff provide feedback on the system's usability. This structured approach reduces bugs and ensures that the final product perfectly aligns with the initial objectives. By combining modern tools with disciplined engineering practices, we guarantee a high-quality delivery.

Conclusion

In conclusion, the proposed Flight Booking System is a comprehensive answer to the technological challenges facing the modern aviation industry. By focusing on the core principles of database integrity—ACID compliance, normalization, and rigorous security—we create a stable foundation for growth. This proposal has detailed how we will tackle issues like double-booking and system latency through advanced concurrency controls and query optimizations. The integration of a distributed design ensures that the system is ready for a global market, providing speed and reliability to users regardless of their location.

We have also prioritized the human element, ensuring that the system is easy to manage for administrators and effortless to use for travelers. The methodology described ensures a low-risk, high-reward development cycle that results in a world-class digital product. As travel continues to evolve, having a flexible and secure database architecture is no longer optional; it is a necessity for survival. This project represents a significant step forward in making air travel booking more transparent, secure, and efficient for everyone involved. We are confident that this system will not only meet the assignment requirements but serve as a viable model for real-world implementation.

References

- Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems (7th Edition). Pearson, 2016.
Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts (7th Edition). McGraw-Hill Education, 2019.
Connolly, T., & Begg, C. Database Systems: A Practical Approach to Design, Implementation, and Management (6th Edition). Pearson, 2015.
- www.workbanch.com
www.oracle.com
www.mysql.com www.

